

“GamePython” Interface
CPSY0950 Final Project
Report on Structure and AI Usage
Madison Sineff and Pearl Chia

Introduction

For this project, we used Cursor to write, develop, and change our code. The AI agent software in Cursor proved to be incredibly useful when needed help with general questions, understanding concepts from tutorials, figuring out how to implement edits specifically to our code (since google searches tend to be more general and may not work for the code structure we already had) and debugging. However, we dedicated a significant amount of time trying to develop and craft our projects based on basic Pygame and other python gaming tutorials that we found online, doing general google searches on potential functions/packages to use in a given situation, as well as using what we learned in class (ex. Matrices, colors, functions) regarding Python to develop the baseline structures necessary. Although the AI agent in cursor effectively processes English requests into applicable code language, we made sure to always review any edits that the AI agent was proposing before accepting as well as asking follow up questions in the chat to understand WHY it was suggesting the changes it did. This helped us not only expand our knowledge on how to structure code in Python effectively, but also pushed us to then become more creative in our computational thinking skills to be able to write new code building off of what AI suggested and using similar formats/ideas when building other game scripts.

Main Hub

As opposed to our previous project that lived in one script, our current project integrates multiple scripts that can launch/close while being supported by our “Main Hub” (or home screen) script that essentially stays open so players can pick games, leave a game when it closes, and pick another. The hub will only close when the user chooses to close the window, not when individual games are closed – this way users can easily move between games without having to relaunch the app every time. Almost everything lives in an App class: setup in `__init__`, the main loop in `run`, rendering in `draw`, and small helpers for music, subprocesses and button graphics. `Self.games` essentially acts as a list of dictionaries in which each entry names a game title and directs the opening of the related script. Primarily, the MainHub uses `launch_game` to launch specific games as opposed to importing them as a library so that each game is a separate process using the same home hub. While the hub is running, the selected background music is played and will stop when a new game is launched but return when a game window has been closed and the user returns to the hub. The drawing portion of the code essentially deals with the main visuals of the hub (draws the title, controls the buttons, includes downloaded images we used for buttons, etc). For this script, we mainly used the AI agent to make little tweaks as we went along with using what we had learned from previous tutorials on using PyGame. For example, if we were writing a code and it came up with an error that we were confused about, we would copy the line

into the agent chat and ask how we could troubleshoot; the agent would often give multiple suggested solutions, from which we would look through and analyze would be the best option for our code before implementing. The agent was also particularly helpful in making various positioning changes that we needed to tweak constantly. For example, once we wrote the code for drawing buttons on the screen, if we needed them to be bigger/smaller/more spread out from each other, we could prompt the agent to make these visual changes and test out the results. This was especially helpful in learning how to make the “zoom” code that helped determine how zoomed out the images we used for buttons were so they could be scaled to a size that was more visually appealing. We also used the agent for general troubleshooting and explaining how to make sections of code more concise.

Anagrams

The Anagrams game is a classic timed word game in which users are presented 6 scrambled letters and have to come up with as many words as possible (by typing into the computer) to gain points. We generated 5 possible scrambles and inputted all of the possible words that the system is constantly checking user guesses against to determine 1) whether guesses are valid words 2) whether words fit the length requirement (3-6 letters) and 3) whether the user guesses a repeat, in which case this would not count. We establish constants (such as minimum word length, max types length, etc.), the list of puzzle objects (the 5 scrambles that we established and all of their possible solutions), and music helpers that that run+loops the background music while the game script is open. The core class is AnagramsGame which acts as the overarching control center for the pygame setup, the game state machine (which is constantly updated), drawing visuals, user input, reactive scoring, and returning the MainHub when selected. For the main logic of the game, we implemented a function that randomly picks an entry from the scramble database. We then begin a timed round, which automatically resets the score, picks a new scramble, and starts recording ticks. The `_submit_guess` section is responsible for organizing responses into their necessary categories (valid/invalid) and then assigning points where available. Finally, the main loop (run) just controls the actual timing of the game event and will flip to the “game over” screen with various options when necessary. For this section, we primarily used AI to help us configure drawings and visuals onto the screen. While we were able to implement word scrambles and solutions, general timing, and reaction to user input via the keyboard on our own as the base, we used the AI agent to help us make further modifications on existing code to make the interface more aesthetically pleasing and more concise. Specifically, the AI agent helped implement code for various buttons on screen as well as the formation of a wordbank when prompted. When we realized that the words were stacking up just on the screen and not specifically in the word bank, we consulted the AI for troubleshooting and it helped implement code that would essentially wrap the text within the word bank, and add words to the word bank matrix horizontally instead of vertically. Further, as with the other games beyond just Anagrams, the agent was extremely helpful in adding small details that helped enhance the overall user experience after the main code was already in place; for example, this could be developing and

explaining code on how to get the color of a button to change upon hovering with a mouse, adding updating text that gives the status of the user's move, Thus, with small tweaks, the AI agent was incredibly helpful especially since we could build off of previous prompting if we just wanted to make a small change to a previous suggestion, as opposed to looking up troubleshooting individually.

Cup Pong

Cup Pong is a classic GamePigeon game in which two players compete to clear their stack of cups first, by throwing ping pong balls inside of them. Although in the original game, players can send the game back and forth to each other, switching between player 1 and player 2, because this game is on a shared computer interface, we had to change the setup so that the two-player game was in a side-by-side view (with both lanes essentially always visible) on a pseudo 3D table layout (trapezoid). Because this game is more complex by nature (involving throwing mechanics and physical matching of the ball to cups on the screen), we used the AI agent as an initial consultant to see the best way to organize the code as a whole, from which we would eventually develop our layered design of both game rules, presentation, and physics running in their own sections (more detail later). Additionally, in the original mobile game, users launch ping pong balls by flicking the screen with their finger; originally, we tried to replicate this by using a dragging mouse motion to direct launching, but we realized that this was difficult to implement into code. We decided to switch to using more of a slingshot mechanism, in which the user must drag the mouse backwards on the ball launching site and use lateral aim (by moving the mouse laterally) to set up their shot; the amount that the ball is dragged back directly corresponds to the power of the shot, which was the primary challenge in using the original "drag forward" mechanism. With this issue in particular, we used the AI agent to help implement the drag back software which would communicate accordingly with users clicking down on the mouse and going backwards. Cup Pong was definitely more complicated than previous scripts because it required lots of physics code to replicate 3D motion on a 2D screen. We relied on the AI agent for developing and suggesting code we could use for the physics portion, since we were relatively unfamiliar with setting this up, and learned a lot about how physics concepts (like gravity, velocity, position) can both be coded, tracked, and applied to the game "world" we created – which was super interesting to see! With this, we ended up developing a layered design with rules, presentation, and physics regarding the game. The CupPongGame portion uses two boolean lists related to player 1 vs. 2 cups which essentially keeps track of the current player, attempts per turn, winner, hits, misses, and the implementation of our own spin on the game: a penalty cup (we came up with this idea ourselves, because we thought it would be fun to add our own twist, in which if one player makes both of their shots, the other player gets an extra cup added to their tack). The AI agent was helpful in adding this penalty cup condition to the already existing loops we had for keeping track of cups and launches. The rack_world_placements portion is responsible for essentially building the world with the cups and table in 2D space. Finally, we also used a GUI structure dataclass to hold mutable loop states as opposed to a long

list of nonlocal variables which would be redundant. Overall, this code ended up being the longest and most complex because the gameplay was far more complicated than a classic 2D minigame, as we had to replicate real world 3D physics into the interface. AI was not only helpful in teaching us how to code physics applicable to game, but was also very essential as an organizer of our code: since there are so many working parts and aspects that we changed/cut out, it was often hard to keep track of all of our code at once, so we would frequently ask the agent if there were any places where we could remove unnecessary code or streamline what we had to keep things as concise and organized as possible.

Connect Four

The Connect Four game, titled “Four in a Row” in our GamePython project, is a two player Python game based on the classic 6-by-7 format. The goal of the game is for either Player 1 (red) or Player 2 (yellow) to successfully place four consecutive—either horizontally, vertically, or diagonally. The `ConnectFourGame` class keeps track of the board, the current player, the winner, whether the game ends in a draw, the most recent move, and the exact cells that created a winning line. AI was helpful when working through the logic of the game because Connect Four requires the program to check several possible win conditions after every move. However, we still had to test the game ourselves to make sure pieces dropped into the correct columns, turns switched properly, full columns could not be played, and the winning cells were correctly identified. The Pygame portion of the Connect Four file handles the actual visual experience of the game, creating the window and board, and keeping track of turns. A challenge with this game was making sure the visual board matched the logic board. For example, the program had to correctly translate a mouse click on the screen into a column number in the code. This required careful coordination between the visual layout and the underlying game logic.

Filler

The Filler game is a two-player color-claiming game based on an 8-by-8 grid. Player 1 starts in the bottom-left corner and Player 2 starts in the top-right corner. Each player then takes turns choosing one of six colors, and their territory expands by absorbing connected cells that match the selected color. The goal is to dominate as much space and cells on the board as possible. This game required us to think carefully about how to represent the board in code. The file uses one grid to store the visible color of each cell and another grid to track which player owns each cell—or if it’s neutral. The most important part of the game logic is the flood-fill algorithm. When a player chooses a new color, all of their currently owned cells change to that color, and then the game checks neighboring cells to see whether any matching color cells can be claimed. AI was helpful for understanding how to structure this type of algorithm, especially because the territory-expansion logic is different from a more straightforward game like Connect Four. However, we still needed to understand how the board, colors, ownership grid, and player turns worked together. We also had to test the game repeatedly to make sure players could not choose invalid colors, the territory expanded correctly, the score updated properly, and the game ended

when all cells on the board were claimed by a player. The Pygame interface for Filler also required design decisions. Since Filler is very visual, spacing and color placement were very important and had to be debugged via trial and error. We had to make sure the grid was centered, the cells were easy to distinguish, and the side panels and instructions were readable. There was initially a struggle with overlapping color choices and two layers of instructions on the bottom after we adjusted window sizing between all minigames to be standard, but we trialed and errored it until it all fit perfectly.

Conclusion

Overall, Cursor and AI assistance played an important role in the development of this project, but they did not replace our own work or understanding. We used AI as a support tool for organizing code, explaining unfamiliar concepts, and debugging. However, the process still required a significant amount of time and problem-solving on our part.

One of the biggest lessons we learned was that AI-generated code is not automatically finished or correct. Many suggestions had to be changed, tested, or rewritten so they would actually work with our file structure and game logic, as well as design. This project helped us to better understand how to use AI ethically and productively; instead of treating Cursor as something that could complete the whole project for us, we used it more like a coding assistant. Through this process, we gained more experience with Python and Pygame, and we were able to truly understand the structure of game design behind every game. Building GamePython helped us to develop a stronger understanding of how multi-file Python projects work and how AI can be used responsibly as part of a larger creative and technical workflow.